SHAP expliqué pas à pas

SHAP par alerte (page Détection) et SHAP par scénario (page SHAP) : tous les calculs, avec des exemples chiffrés

Comment lire ce document. Chaque partie suit le même schéma : (1) l'idée en une phrase, (2) un exemple chiffré que tu peux refaire à la main, (3) la correspondance exacte avec ton code. Les chiffres des exemples sont **inventés pour apprendre** : ils ne viennent pas de ta base. Les noms de fonctions et de lignes viennent de ton code (*backend/app/domains/transmission/service.py* et *backend/scripts/shap_alertes.py*).

Sommaire

- **1.** Le vocabulaire (ligne, épisode, alerte, feature, scénario, score)
- **2.** Ce qu'est SHAP, avec un calcul de Shapley fait à la main
- **3.** SHAP par alerte (page Détection) : le calcul complet, étape par étape
- **4.** Du SHAP de l'alerte à la recommandation (fonction estimer)
- **5.** SHAP par scénario (page SHAP) : le calcul complet, étape par étape
- **6.** Comparaison des deux calculs
- **7.** Limites et précautions à écrire dans le rapport
- **8.** Aide-mémoire : où est quoi dans le code

1. Le vocabulaire

Mot	Ce que c'est	Chiffre dans ton projet
Ligne	Une mesure du jeu de données : température frigo, thermostat, météo, etc.	1 ligne = **5 secondes** (STEP_MINUTES = 5/60). 12 lignes = 1 minute.
Feature (mesure calculée)	Une colonne que le modèle regarde. Certaines sont brutes (fridge_temp), la plupart sont calculées sur une fenêtre glissante (fridge_std_5min = écart-type sur 5 min, fridge_repeat_2min = séquence de 2 min déjà vue).	Fabriquées par la fonction engineer. Notées feats dans le code.
Score	La sortie du modèle pour une ligne : plus il est grand, plus le modèle pense « attaque ».	Calculé pour **chaque ligne** (toutes les 5 s).
Épisode	Une **vraie attaque** injectée par toi dans la simulation (la vérité).	Dure 15 à 25 min = 180 à 300 lignes (EP_MIN, EP_MAX).
Alerte	Une suite **continue de lignes en alarme** du modèle. C'est ce que le modèle croit voir. Fonction alert_events.	Durée variable : elle peut être plus courte que l'épisode, coupée en plusieurs, ou absente.
Scénario	Un **type** d'attaque injectée : MITM_gel, MITM_rejeu, Injection_biais_1.0, Injection_derive, etc.	Liste = ATTACK_NAMES.

Alerte et épisode ne sont pas la même chose. Exemple : un épisode de 20 min (10h00-10h20). Le modèle sonne de 10h03 à 10h18. L'épisode dure 20 min, l'alerte 15 min. Le SHAP par alerte travaille sur les 15 min de l'alerte, pas sur les 20 min de l'attaque.

2. Ce qu'est SHAP

L'idée en une phrase. Le modèle est une boîte noire : il donne un score mais pas la raison. SHAP répartit l'écart entre *le score de cette ligne* et *le score moyen du modèle* entre toutes les features : chacune reçoit une part, la valeur SHAP.

Formule (à retenir)
score de la ligne = score moyen + SHAP(feature 1) + SHAP(feature 2) + ... + SHAP(feature n)

SHAP > 0 : la feature pousse vers « attaque ». SHAP < 0 : elle pousse vers « normal ». Plus la valeur absolue est grande, plus la feature a pesé.

2.1 Exemple : le calcul de Shapley avec 2 features

SHAP est basé sur les valeurs de Shapley (théorie des jeux). Le principe : on **ajoute les features dans tous les ordres possibles** et on regarde de combien le score augmente quand une feature arrive. On fait la moyenne de ces augmentations.

Prenons deux features A = fridge_std_5min et B = fridge_mean_5min, et un modèle qui donne ces scores selon les features connues :

Features connues	Score du modèle
aucune (score moyen)	0,10

Features connues	Score du modèle
A seule	0,50
B seule	0,20
A et B	0,90

Contribution de A. Deux ordres possibles :

- Ordre A puis B : A arrive en premier, le score passe de 0,10 à 0,50, gain = 0,50 - 0,10 = **0,40**.
- Ordre B puis A : A arrive après B, le score passe de 0,20 à 0,90, gain = 0,90 - 0,20 = **0,70**.

SHAP(A) = moyenne des gains = (0,40 + 0,70) / 2 = **0,55**.

Contribution de B.

- Ordre B puis A : gain = 0,20 - 0,10 = **0,10**.
- Ordre A puis B : gain = 0,90 - 0,50 = **0,40**.

SHAP(B) = (0,10 + 0,40) / 2 = **0,25**.

Vérification. Score moyen + SHAP(A) + SHAP(B) = 0,10 + 0,55 + 0,25 = **0,90** = score de la ligne. La somme des parts redonne exactement le score : c'est la propriété qui rend SHAP fiable.

2.2 Et avec beaucoup de features ?

Avec n features, il y a n! ordres possibles : le calcul direct est impossible. Ton code utilise **TreeExplainer** : un algorithme qui exploite la structure des arbres (RandomForest, XGBoost, LightGBM...) pour obtenir les **mêmes valeurs exactes** sans énumérer tous les ordres. Il reste coûteux : le commentaire de ton code estime **environ 0,3 seconde par ligne**. C'est la raison pour laquelle tu ne calcules pas SHAP sur toutes les lignes (voir parties 3 et 5).

Unité des valeurs SHAP. Selon le modèle, les valeurs SHAP sont exprimées en probabilité ou en « sortie brute » du modèle (par exemple log-odds pour le boosting). Je n'ai pas vérifié lequel s'applique à ton meilleur modèle. Ce n'est pas gênant pour ton usage : tu utilises surtout le **signe** et le **classement** des features, pas la valeur exacte.

3. SHAP par alerte (page Détection)

Question posée : pour *cette alerte précise*, quelles mesures ont poussé le modèle à sonner ?
Fonction : *alert_shap* (service.py:391). Elle est appelée à la fin de chaque entraînement et, pour un run déjà en base, par le script *shap_alertes.py*.

3.0 Le script shap_alertes.py (rattrapage d'un run existant)

- Il choisit le run : celui donné en argument, sinon le **dernier entraînement terminé** (status = termine).
- Il lit en base les alertes (TransmissionAlert) et le jeu de données (TransmissionDatasetRow) de ce run.
- Il reconstruit un tableau pandas et recalcule les features avec *tm.engineer(df)*.
- **Garde-fou (ligne 62)** : si le modèle en service ou ses features ne correspondent pas au run, il s'arrête sans rien écrire. Expliquer un run avec un autre modèle donnerait des résultats faux.
- Il recharge le modèle (.pkl), appelle *alert_shap*, **efface** les anciennes lignes SHAP du run puis insère les nouvelles dans la table *transmission_alert_shap*. On peut donc le relancer sans doublons.

3.1 Les étapes de alert_shap

Étape 1 : préparer les données.

```
X = df[feats].to_numpy(float)                       # matrice lignes x features
ref = X[df['label'] == 0].mean(axis=0)              # moyenne de chaque feature HORS attaque
explainer = shap.TreeExplainer(model)
```

ref est la « valeur normale » de chaque feature : sa moyenne sur toutes les lignes sans attaque (label = 0). Elle sert de point de comparaison dans l'affichage. Exemple : si fridge_std_5min vaut en moyenne 0,35 quand tout va bien, alors ref(fridge_std_5min) = 0,35.

Étape 2 : choisir les lignes de l'alerte.

```
rows = np.arange(a['start_idx'], a['end_idx'])
if len(rows) > max_rows:                            # max_rows = 6
    rows = rows[np.linspace(0, len(rows)-1, max_rows).astype(int)]
```

Une alerte est la plage *start_idx* à *end_idx* (fin exclue). Si elle a plus de 6 lignes, on en garde **6, réparties régulièrement**.

Exemple chiffré. Alerte de 15 min = 15 x 12 = **180 lignes**, avec start_idx = 1000 et end_idx = 1180. Alors np.arange donne les indices 1000 à 1179, soit 180 valeurs. On calcule np.linspace(0, 179, 6) :

Étape	Calcul	Résultat
Pas régulier	179 / (6 - 1) = 35,8	positions 0 ; 35,8 ; 71,6 ; 107,4 ; 143,2 ; 179
astype(int) arrondit vers le bas	35,8 -> 35 ; 71,6 -> 71 ; 107,4 -> 107 ; 143,2 -> 143	0 ; 35 ; 71 ; 107 ; 143 ; 179
Indices réels dans le jeu de données	1000 + chaque position	**1000 ; 1035 ; 1071 ; 1107 ; 1143 ; 1179**

En temps : une ligne toutes les 5 s, donc environ une ligne tous les 3 minutes. On regarde le début, le milieu et la fin de l'alerte. Si l'alerte a 6 lignes ou moins, on les prend toutes.

Pourquoi seulement 6 lignes ? Temps de calcul. À 0,3 s par ligne : 180 lignes = environ 54 s par alerte, contre environ 2 s pour 6 lignes. Avec des dizaines d'alertes par run, la différence est énorme.

Étape 3 : calculer les valeurs SHAP.

```
values = explainer.shap_values(X[rows])
if isinstance(values, list): values = values[-1]   # on garde la classe 'attaque'
values = np.asarray(values)
if values.ndim == 3: values = values[:, :, -1]     # idem selon la version de shap
```

On obtient un tableau de 6 lignes x (nombre de features). Les deux « if » ne servent qu'à gérer les formats de sortie selon la version de shap : on veut toujours la colonne de la classe « attaque ». Chaque case est la part de la feature (colonne) pour la ligne (rangée).

Exemple avec 4 features (en vrai il y en a beaucoup plus). Tableau des valeurs SHAP :

Ligne	std_5min	mean_5min	repeat_2min	thermostat_temp
1000	+0,52	+0,08	0,00	-0,04
1035	+0,58	+0,05	+0,01	-0,03
1071	+0,61	+0,06	-0,01	-0,05
1107	+0,63	+0,04	0,00	-0,04
1143	+0,60	+0,07	+0,01	-0,03
1179	+0,66	+0,06	-0,01	-0,05

Étape 4 : moyenne avec le signe.

```
m = values.mean(axis=0)     # moyenne par colonne, SANS valeur absolue
x = X[rows].mean(axis=0)    # valeur réelle moyenne de chaque feature pendant l'alerte
```

Calcul à la main, feature par feature :

Feature	Somme des 6 SHAP	Moyenne m (somme / 6)
std_5min	0,52 + 0,58 + 0,61 + 0,63 + 0,60 + 0,66 = 3,60	3,60 / 6 = **+0,60**
mean_5min	0,08 + 0,05 + 0,06 + 0,04 + 0,07 + 0,06 = 0,36	0,36 / 6 = **+0,06**
repeat_2min	0 + 0,01 - 0,01 + 0 + 0,01 - 0,01 = 0,00	0,00 / 6 = **0,00**
thermostat_temp	-0,04 - 0,03 - 0,05 - 0,04 - 0,03 - 0,05 = -0,24	-0,24 / 6 = **-0,04**

Pourquoi garder le signe ici ? On veut savoir ce qui a poussé *vers* l'attaque. Une feature qui pousse vers « normal » (SHAP négatif) n'explique pas l'alerte et doit passer en bas du classement.

Pour *x* (valeur réelle moyenne), prenons std_5min. Valeurs mesurées sur les 6 lignes : 0,02 ; 0,01 ; 0,01 ; 0,01 ; 0,01 ; 0,02. Somme = 0,08, moyenne = 0,08 / 6 = **0,0133**. Et rappel : ref(std_5min) = **0,35**.

Étape 5 : garder le top 6.

```
for rang, i in enumerate(np.argsort(m)[::-1][:top], 1):   # top = 6
    out.append({'feature': feats[i], 'rang': rang,
                'shap': round(m[i], 5), 'valeur': round(x[i], 4),
                'valeur_normale': round(ref[i], 4)})
```

np.argsort(m) trie du plus petit au plus grand, *[::-1]* inverse (du plus grand au plus petit), *[:6]* garde les 6 premiers. Avec nos 4 features d'exemple, voici ce qui est enregistré dans la table (une rangée par feature) :

Rang	Feature	SHAP moyen	Valeur pendant l'alerte	Valeur normale (ref)
1	fridge_std_5min	+0,60	0,0133	0,35
2	fridge_mean_5min	+0,06	6,87	7,00
3	fridge_repeat_2min	0,00	0,00	0,00
4	thermostat_temp	-0,04	24,1	24,0

(Les valeurs de mean_5min, repeat_2min et thermostat_temp sont inventées pour compléter l'exemple.)

Lecture pour l'opérateur. « Cette alerte est surtout due à l'écart-type du frigo sur 5 min : 0,0133 au lieu de 0,35 en temps normal. Le signal ne bouge presque plus (capteur figé ?). » Les autres features pèsent très peu.

Combien de lignes en base ? 6 lignes par alerte (rang 1 à 6). Un run de 40 alertes écrit donc 40 x 6 = **240 lignes** dans transmission_alert_shap, ce que le script affiche à la fin. Le libellé français de chaque feature est ajouté par feature_label.

4. Du SHAP de l'alerte à la recommandation

Cette partie est dans *frontend/src/pages/etude/recommandations.ts*. Le modèle ne dit que « attaque » ou « normal ». La **famille** d'attaque est donc une **estimation** déduite des features que SHAP a classées en tête.

4.1 Quelle famille pour chaque feature (fonction familleDe)

Si le nom de la feature contient...	Famille	Signification
repeat_2min	rejeu	Séquence déjà transmise, répétée
std_, absdiff_, diff_1	fige	Variation très faible : signal figé ou trop régulier
mean_, min_, max_, q10_, q90_, _temp, diff	niveau	Valeur décalée ou qui dérive
autre	indetermine	Pas de lecture possible

4.2 La fonction estimer : trois règles, dans cet ordre

Règle 1 (priorité au rejeu). Si une feature repeat_2min a un SHAP positif et une valeur pendant l'alerte plus grande que sa valeur normale, la famille est **rejeu**. Confiance = 1 - (valeur normale / valeur).

Règle 2 (priorité au figé). Si une feature std ou absdiff a un SHAP positif, une valeur normale > 0 et une valeur pendant l'alerte **inférieure ou égale à 25 % de sa valeur normale**, la famille est **fige**. Confiance = 1 - (valeur / valeur normale).

Règle 3 (vote pondéré). Sinon, on additionne les SHAP **positifs** de chaque famille. La famille qui pèse au moins 50 % du total gagne ; sinon la réponse est **indéterminé** (on ne force pas).

Application à notre exemple de la partie 3 :

Test	Calcul	Résultat
Règle 1 : repeat_2min a un SHAP > 0 ?	SHAP = 0,00, pas > 0	Non, on passe à la règle 2
Règle 2 : std_5min a un SHAP > 0 ?	+0,60 > 0	Oui
Valeur normale > 0 ?	0,35 > 0	Oui
Valeur <= 25 % de la normale ?	25 % x 0,35 = 0,0875 ; 0,0133 <= 0,0875	**Oui : famille fige**
Confiance	1 - (0,0133 / 0,35) = 1 - 0,038	**0,962 soit environ 96 %**

Exemple du vote (règle 3), pour comprendre. Supposons qu'aucune règle prioritaire ne s'applique et que les SHAP positifs soient : famille niveau = 0,30 + 0,10 = 0,40 ; famille fige = 0,10 ; famille rejeu = 0. Total = 0,50. Part de niveau = 0,40 / 0,50 = 80 % >= 50 % : la famille est **niveau**, confiance 0,80. Si les parts étaient 45 % / 40 % / 15 %, aucune n'atteint 50 % : réponse **indéterminé**.

4.3 Les recommandations

Selon la famille, le rapport propose des contrôles spécifiques (par exemple pour fige : vérifier que le capteur et la liaison ne sont pas bloqués, comparer avec le second capteur), en plus d'étapes générales Immédiat, Vérification et Suivi.

Pourquoi « estimation, pas certitude ». (1) Le modèle ne connaît pas le type d'attaque. (2) La règle peut répondre « indéterminé ». (3) Pour tes données simulées tu peux la vérifier : tu connais le vrai scénario (FAMILLE_REELLE) et tu mesures le pourcentage de bonnes estimations. En production, cette vérité n'existe pas : le texte doit rester une **hypothèse**.

5. SHAP par scénario (page SHAP)

Question posée : quelles mesures le modèle regarde-t-il le plus, en général, puis pour chaque type d'attaque ? **Fonction :** *_shap_summary* (service.py:794). Elle alimente les deux graphiques de la page ShapPage.tsx : « global » et « par scénario » (boutons de choix du scénario).

5.1 Les étapes de _shap_summary

Étape 1 : choisir les lignes d'attaque.

```
y = df['label'].to_numpy()
types = df['attack_type'].to_numpy()
rows = np.concatenate([val_idx[y[val_idx] == 1], test_idx[y[test_idx] == 1]])
```

On ne garde que les lignes **réellement attaquées** (label = 1) de la **validation** et du **test**. On exclut l'entraînement, car le modèle y a été ajusté : l'expliquer là donnerait une image trop optimiste.

Étape 2 : tirer 300 lignes au hasard.

```
rng = np.random.default_rng(tr.SEED)
rows = np.sort(rng.choice(rows, min(300, len(rows)), replace=False))
```

S'il y a plus de 300 lignes attaquées, on en tire **300 sans remise**. S'il y en a moins, on les prend toutes (le min). La graine fixe (SEED) rend le tirage **reproductible** : deux exécutions donnent le même résultat. Raison du plafond : le coût d'environ 0,3 s par ligne, soit environ 90 s pour 300 lignes.

Étape 3 : calculer SHAP sur ces lignes. Même code que la partie 3 : on obtient un tableau 300 lignes x features.

Étape 4 : valeur absolue.

```
imp = np.abs(values)
```

Ici on mesure l'**importance**, pas la direction. Un +0,5 et un -0,5 pèsent autant. Sans la valeur absolue, ils se compenseraient : (+0,5 + (-0,5)) / 2 = 0, et une feature très importante paraîtrait inutile. Avec la valeur absolue : (0,5 + 0,5) / 2 = **0,5**.

Étape 5 : moyenne et top, global puis par scénario.

```
def top(mask, k=6):
    m = imp[mask].mean(axis=0)              # moyenne des |SHAP| des lignes du masque
    order = np.argsort(m)[::-1][:k]           # tri décroissant, on garde k
    return [{'feature':..., 'importance': round(m[i], 4)} for i in order]

out['global'] = top(tout, 15)                 # toutes les lignes, 15 features
for nm in ATTACK_NAMES:
    mask = types[rows] == nm                  # seulement les lignes de CE scénario
    if mask.any(): out['par_scenario'][nm] = top(mask, 6)
```

Le **masque** est un filtre : un tableau vrai/faux qui sélectionne les lignes dont le type d'attaque est le scénario voulu. *if mask.any()* évite de produire un graphique pour un scénario absent de l'échantillon.

5.2 Exemple chiffré complet

Mini-échantillon de 5 lignes d'attaque : 3 du scénario **gel** et 2 du scénario **rejeu**, avec 2 features. Tableau des |SHAP| (valeurs absolues déjà prises) :

Ligne	Scénario	|SHAP| std_5min	|SHAP| repeat_2min
1	gel	0,55	0,02
2	gel	0,65	0,01
3	gel	0,60	0,03
4	rejeu	0,05	0,50
5	rejeu	0,07	0,54

Graphique « global » (toutes les 5 lignes) :

- std_5min : (0,55 + 0,65 + 0,60 + 0,05 + 0,07) / 5 = 1,92 / 5 = **0,384**
- repeat_2min : (0,02 + 0,01 + 0,03 + 0,50 + 0,54) / 5 = 1,10 / 5 = **0,220**

Graphique « gel » (lignes 1 à 3) :

- std_5min : (0,55 + 0,65 + 0,60) / 3 = 1,80 / 3 = **0,600**
- repeat_2min : (0,02 + 0,01 + 0,03) / 3 = 0,06 / 3 = **0,020**

Graphique « rejeu » (lignes 4 et 5) :

- std_5min : (0,05 + 0,07) / 2 = 0,12 / 2 = **0,060**
- repeat_2min : (0,50 + 0,54) / 2 = 1,04 / 2 = **0,520**

Graphique	std_5min	repeat_2min	Ce qu'on apprend
Global	0,384	0,220	Les deux comptent, rien de net
Gel	0,600	0,020	Le gel se reconnaît à la variation (std)
Rejeu	0,060	0,520	Le rejeu se reconnaît à la séquence répétée

Pourquoi le graphique par scénario est utile. Le global mélange tout et cache les différences. Le graphique par scénario montre que **chaque type d'attaque s'appuie sur des features différentes, cohérentes avec sa physique** : un gel fait chuter la variation, un rejeu fait monter la répétition. C'est la preuve que le modèle a appris des raisons sensées, et cela justifie les règles de familleDe dans recommandations.ts.

6. Comparaison des deux calculs

	SHAP par alerte (page Détection)	SHAP par scénario (page SHAP)
Fonction	alert_shap (service.py:391)	_shap_summary (service.py:794)
Question	Pourquoi CETTE alerte a-t-elle sonné ?	Quelles features comptent pour détecter chaque type d'attaque ?
Lignes utilisées	6 lignes maximum par alerte, réparties régulièrement (np.linspace)	300 lignes d'attaque tirées au hasard dans validation + test
Quelles lignes ?	Lignes de l'alerte (que l'attaque soit vraie ou fausse alarme)	Seulement des lignes réellement attaquées (label = 1)
Valeur absolue ?	**Non** : on garde le signe (direction)	**Oui** : on mesure l'importance
Moyenne sur	Les 6 lignes d'une alerte	Les lignes d'un scénario (ou toutes pour le global)
Résultat	Top 6 features qui poussent vers l'attaque, avec valeur et valeur normale	Top 15 global, plus top 6 par scénario, avec importance moyenne
Utilité	Expliquer l'alerte à l'opérateur et produire la recommandation	Bilan statistique pour valider le modèle et le rapport
Stockage	Table transmission_alert_shap	Résumé du run, lu par la page SHAP

Un point commun : les deux utilisent le même explainer (TreeExplainer) et les deux **échantillonnent** pour limiter le temps de calcul.

7. Limites et précautions à écrire dans le rapport

- **Échantillonnage.** 6 lignes par alerte et 300 lignes au total : c'est une approximation du SHAP complet, choisie pour le temps de calcul (environ 0,3 s par ligne). Les lignes sont réparties régulièrement pour couvrir toute l'alerte.
- **Alerte et attaque sont différentes.** SHAP par alerte explique ce que le modèle a vu pendant l'alerte, pas toute l'attaque.
- **Features corrélées.** Des features proches (std_5min, std_15min) se partagent le crédit : l'importance d'une peut paraître plus faible que sa vraie utilité. Lire les résultats par familles de features est plus sûr que feature par feature.
- **SHAP explique le modèle, pas la réalité.** Il dit ce que le modèle a utilisé, pas la cause physique de l'attaque.
- **La famille est une estimation.** Elle est vérifiée sur les attaques injectées (FAMILLE_REELLE), mais en production il n'y a pas de vérité pour la valider.
- **Unités.** Selon le modèle, les valeurs SHAP sont en probabilité ou en sortie brute : le classement et le signe sont fiables, la valeur absolue se compare seulement à l'intérieur d'un même modèle.

8. Aide-mémoire : où est quoi dans le code

Élément	Fichier	Rôle
alert_shap	backend/app/domains/transmission/service.py:391	SHAP par alerte (top 6, 6 lignes)

Élément	Fichier	Rôle
_shap_summary	backend/app/domains/transmission/service.py:794	SHAP global et par scénario (300 lignes)
feature_label	backend/app/domains/transmission/service.py:206	Nom français lisible d'une feature
study_records	backend/app/domains/transmission/service.py:424	Construit les alertes puis appelle alert_shap
alert_events	backend/scripts/transmission_alertes.py:83	Transforme l'alarme ligne par ligne en alertes [début, fin)
shap_alertes.py	backend/scripts/shap_alertes.py	Rattrape le SHAP des alertes d'un run déjà en base
estimer, familleDe	frontend/src/pages/etude/recommandations.ts	Famille estimée et confiance à partir du SHAP
ShapPage	frontend/src/pages/etude/ShapPage.tsx	Affiche le global et le par scénario
DetectionPage	frontend/src/pages/etude/DetectionPage.tsx	Affiche les alertes et leurs explications

Phrase de synthèse pour ton rapport. « Le score du modèle est calculé à chaque ligne de 5 secondes. Pour chaque alerte, nous calculons les valeurs SHAP (TreeExplainer) sur 6 lignes réparties sur sa durée, nous les moyennons avec leur signe et retenons les 6 features qui poussent le plus vers l'attaque. Pour le bilan par scénario, nous calculons SHAP sur 300 lignes d'attaque tirées au hasard dans la validation et le test et moyennons les valeurs absolues. La famille d'attaque est estimée à partir de ces features et vérifiée sur les scénarios injectés. »